

同济大学计算机系

计算机组成原理课程综合实验报告



学 号 2351520

姓 名 毛星博

专 业 计算机科学与技术

授课老师 陈永生

一、实验内容

本次实验，将会对给出的 31 条 MPIS 指令进行功能分析、数据通路设计，根据指令流程图和操作时间表设计控制部件，并使用 verilogHDL 语言实现 31 条 MPIS 指令的 CPU 设计，并对其进行前仿真、后仿真、下板运行等测试。

二、CPU 数据通路设计

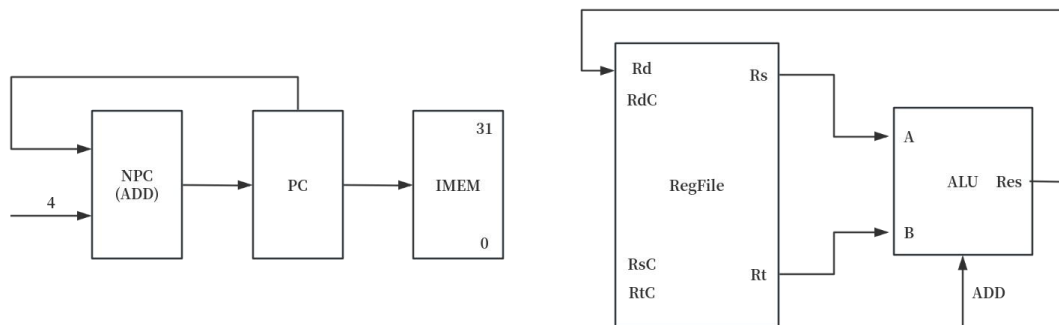
(1) 根据每条指令的功能，确定执行过程中所用部件和各部件的数据输入来源。

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
指令	PC	IMEM	Decoder	RegFile	Rd	A	B	ALUC3[3:0]	DM_R	DM_L	DM_W	Addr	Data	EXT1	EXT5	EXT16	EXT16 (signed)	EXT10 (signed)	A	B
1 ADD	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	ADD00010												
2 ADDU	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	ADD00000												
3 SUB	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	SUB00011												
4 SUBU	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	SUB00001												
5 AND	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	AND01000												
6 OR	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	OR01011												
7 XOR	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	XOR01101												
8 INCOR	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	XOR01111												
9 SLT	PC+4	PC	IMEM	1	EXT1	RegFile-Rs	RegFile-Rt	SLT10011												
10 SLTU	PC+4	PC	IMEM	1	EXT1	RegFile-Rs	RegFile-Rt	SLTU10101												
11 SLE	PC+4	PC	IMEM	1	ALU	EXT5	RegFile-Rs	SLE11101												
12 SLEU	PC+4	PC	IMEM	1	ALU	EXT5	RegFile-Rs	SLEU11011												
13 SRL	PC+4	PC	IMEM	1	ALU	EXT5	RegFile-Rs	SRL11011												
14 SRA	PC+4	PC	IMEM	1	ALU	EXT5	RegFile-Rs	SRA11001												
15 SLAV	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	SLE11101												
16 SRLV	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	SRL11011												
17 SRAV	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	SRA11001												
18 J	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	SRA11001												
19 JAL	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	SRA11001												
20 JALR	PC+4	PC	IMEM	1	ALU	RegFile-Rs	RegFile-Rt	SRA11001												
21 ADDI	PC+4	PC	IMEM	1	ALU	RegFile-Rs	EXT16(signed)	ADD00010												
22 ADDUI	PC+4	PC	IMEM	1	ALU	RegFile-Rs	EXT16(signed)	ADD00000												
23 ANDI	PC+4	PC	IMEM	1	ALU	RegFile-Rs	EXT16	AND01000												
24 ORI	PC+4	PC	IMEM	1	ALU	RegFile-Rs	EXT16	OR01011												
25 XORI	PC+4	PC	IMEM	1	ALU	RegFile-Rs	EXT16	XOR01101												
26 LW	PC+4	PC	IMEM	1	DMEM	RegFile-Rs	EXT16(signed)	ADD00000												
27 SW	PC+4	PC	IMEM	0	DMEM	RegFile-Rs	EXT16(signed)	ADD00000												
28 BEQ	PC+4	PC	IMEM	0	RegFile-Rs	RegFile-Rt	SUBU00001													
29 BNE	PC+4	PC	IMEM	0	RegFile-Rs	RegFile-Rt	SUBU00001													
30 SLTI	PC+4	PC	IMEM	1	EXT1	RegFile-Rs	EXT16(signed)	SLT10011												
31 SLTIU	PC+4	PC	IMEM	1	EXT1	RegFile-Rs	EXT16(signed)	SLTU10101												
J	PC+4	PC	IMEM	1	ALU	RegFile-Rs	EXT16	LUR10000												
JAL	PC+4	PC	IMEM	1	PC+5															

(2) 根据每条指令所涉及部件和部件的数据输入来源，画出每条指令的数据通路，在已形成的数据通路下，画出每条指令从取指到执行过程的指令流程图。

R 型指令

1 ADD



PC -> IMEM

PC + 4 -> NPC

NPC -> PC

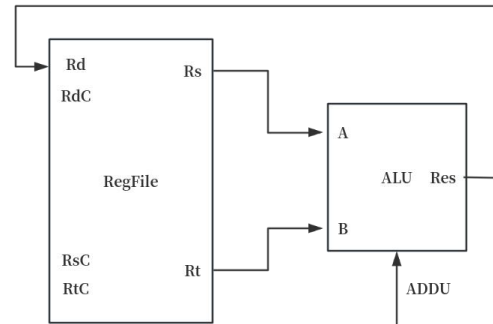
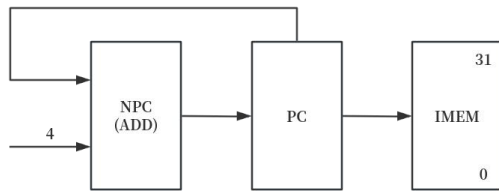
Rs -> A

Rt -> B

(A + B -> Res)

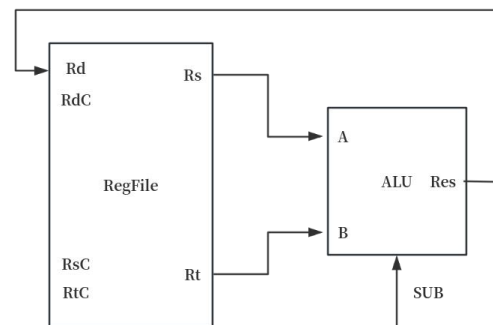
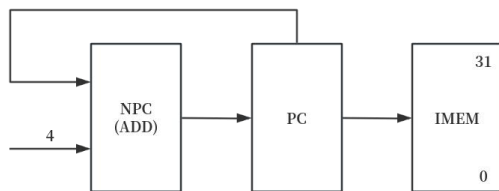
Res -> Rd

2 ADDU



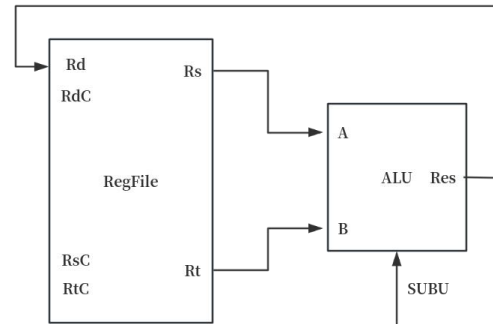
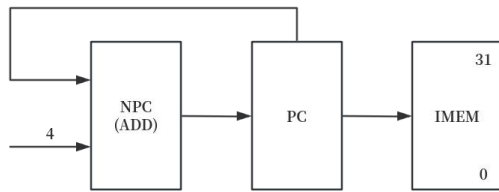
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs -> A
 Rt -> B
 (A + B -> Res)
 Res -> Rd

3 SUB



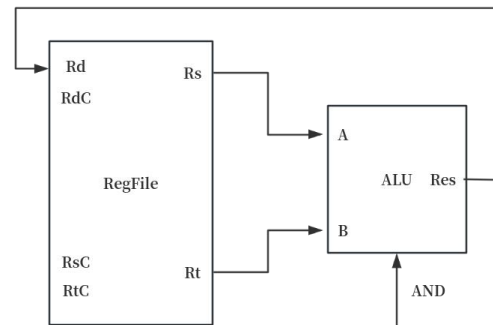
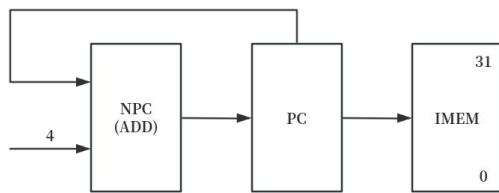
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs -> A
 Rt -> B
 (A - B -> Res)
 Res -> Rd

4 SUBU



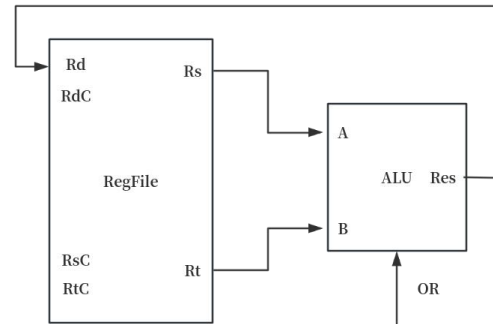
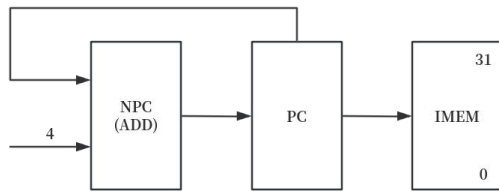
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs -> A
 Rt -> B
 (A - B -> Res)
 Res -> Rd

5 AND



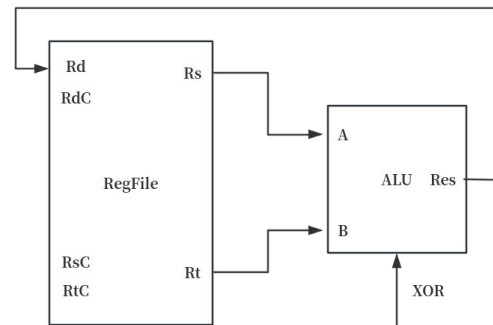
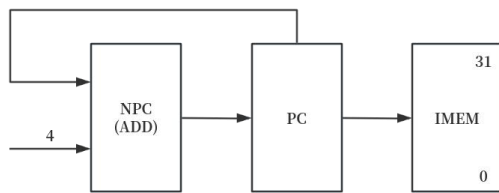
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs -> A
 Rt -> B
 (A & B -> Res)
 Res -> Rd

6 OR



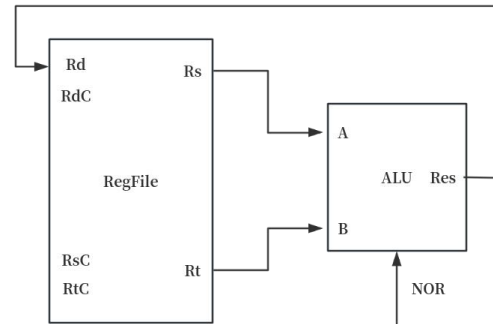
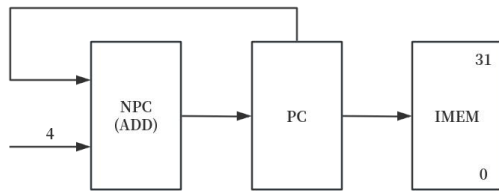
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs -> A
 Rt -> B
 (A | B -> Res)
 Res -> Rd

7 XOR



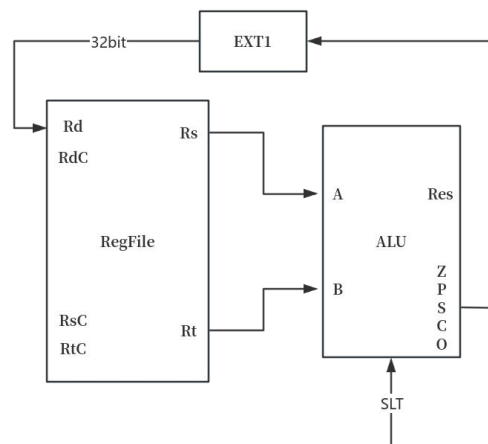
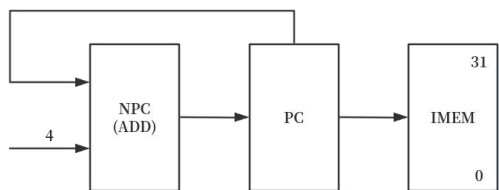
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs -> A
 Rt -> B
 (A $\oplus$ B -> Res)
 Res -> Rd

8 NOR



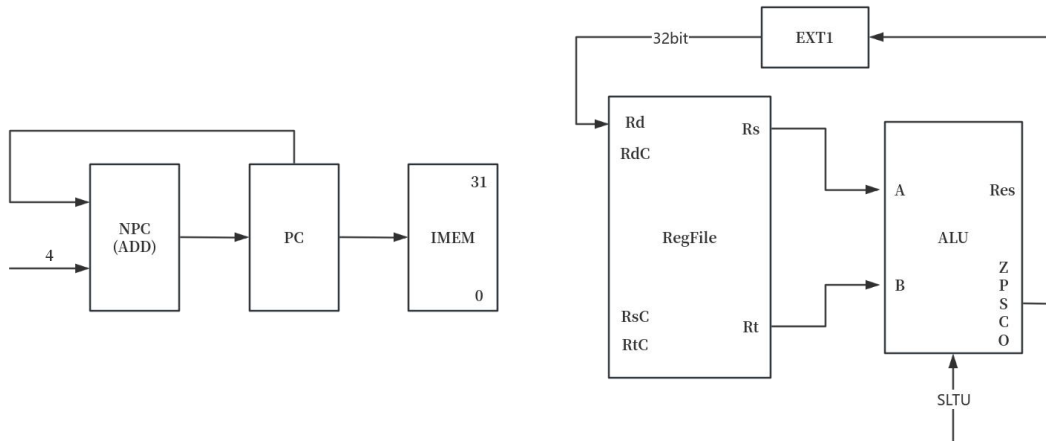
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs -> A
 Rt -> B
 $(A \odot B \rightarrow \text{Res})$
 Res -> Rd

9 SLT



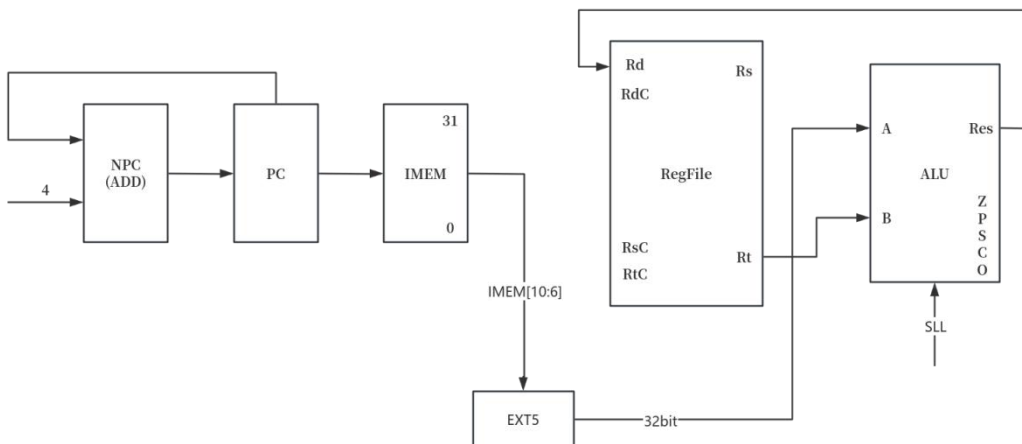
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs -> A
 Rt -> B
 $(A - B \rightarrow \text{Res})$
 SF -> EXT1
 EXT1 -> Rd

10 SLTU



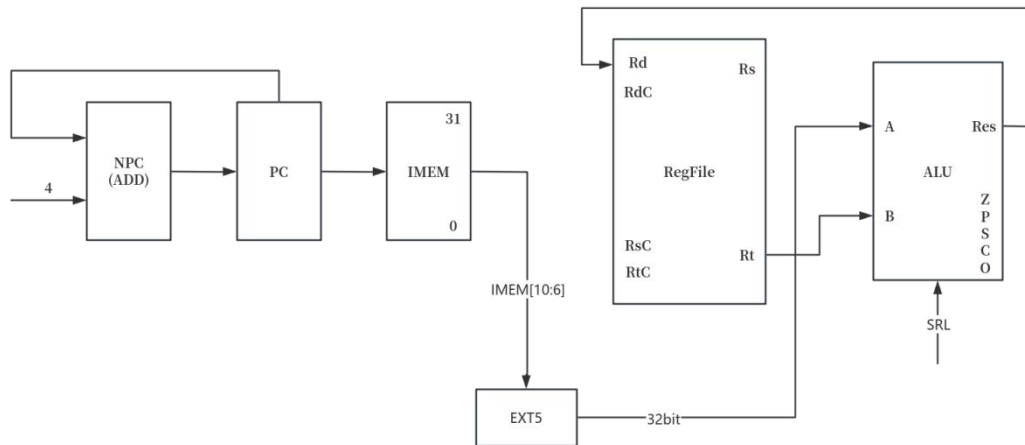
PC $\rightarrow$ IMEM
 PC + 4 $\rightarrow$ NPC
 NPC $\rightarrow$ PC
 Rs $\rightarrow$ A
 Rt $\rightarrow$ B
 (A - B $\rightarrow$ Res)
 SF $\rightarrow$ EXT1
 EXT1 $\rightarrow$ Rd

11 SLL



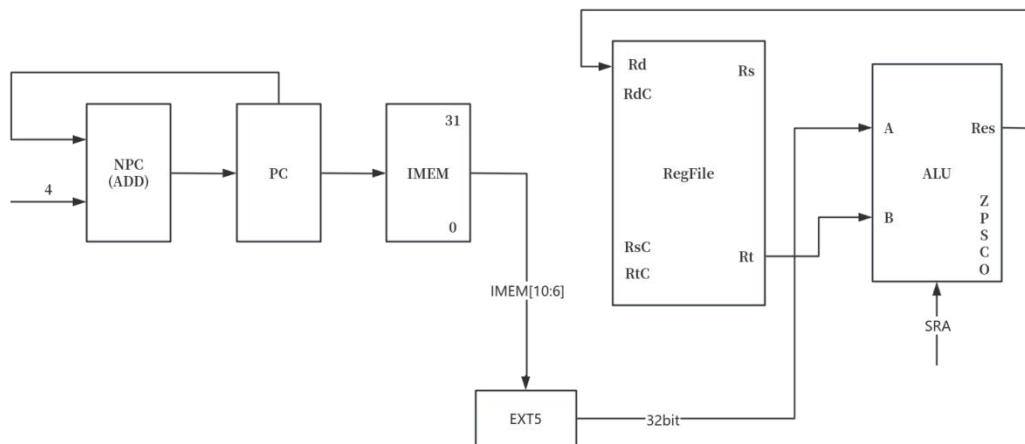
PC $\rightarrow$ IMEM
 PC + 4 $\rightarrow$ NPC
 NPC $\rightarrow$ PC
 IMEM[10:6] $\rightarrow$ EXT5
 EXT5 $\rightarrow$ A
 Rt $\rightarrow$ B
 (B $\ll$ A $\rightarrow$ Res)
 Res $\rightarrow$ Rd

12 SRL



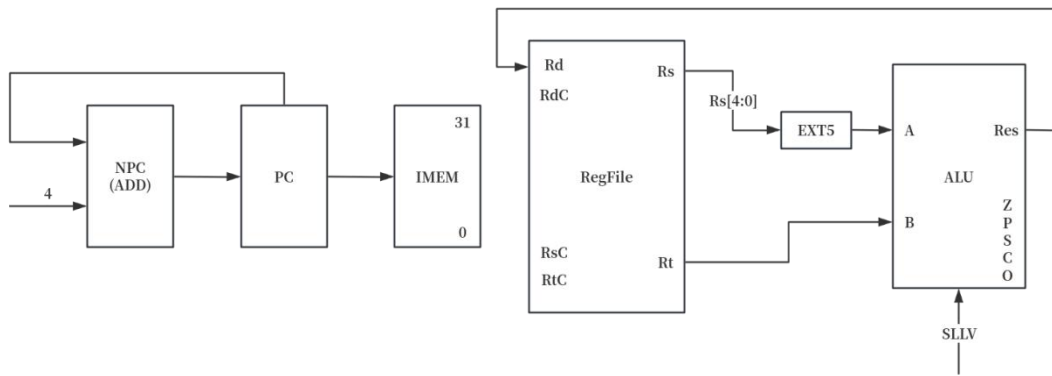
PC -> IMEM
PC + 4 -> NPC
NPC -> PC
IMEM[10:6] -> EXT5
EXT5 -> A
Rt -> B
(B >>> A -> Res)
Res -> Rd

13 SRA



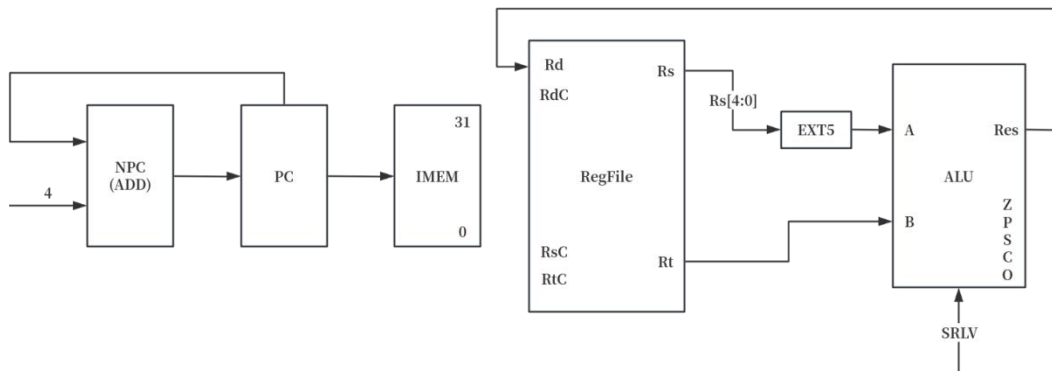
PC -> IMEM
PC + 4 -> NPC
NPC -> PC
IMEM[10:6] -> EXT5
EXT5 -> A
Rt -> B
(B >> A -> Res)
Res -> Rd

14 SLLV



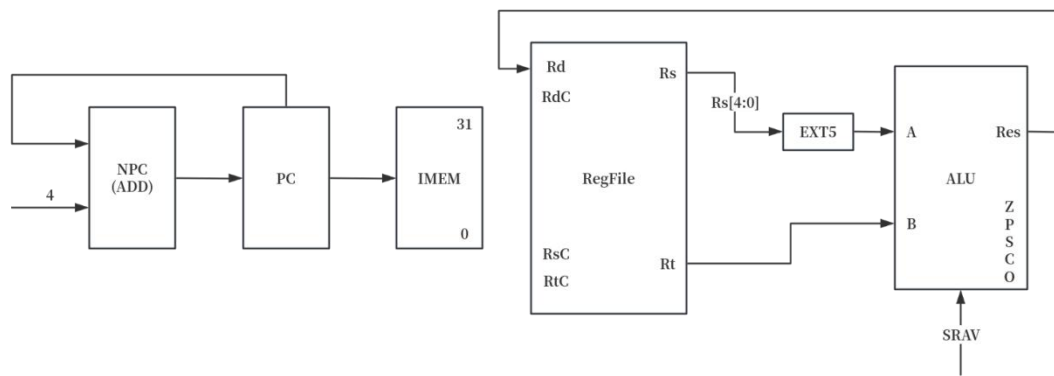
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs[4:0] -> EXT5
 EXT5 -> A
 Rt -> B
 (B << A -> Res)
 Res -> Rd

15 SRLV



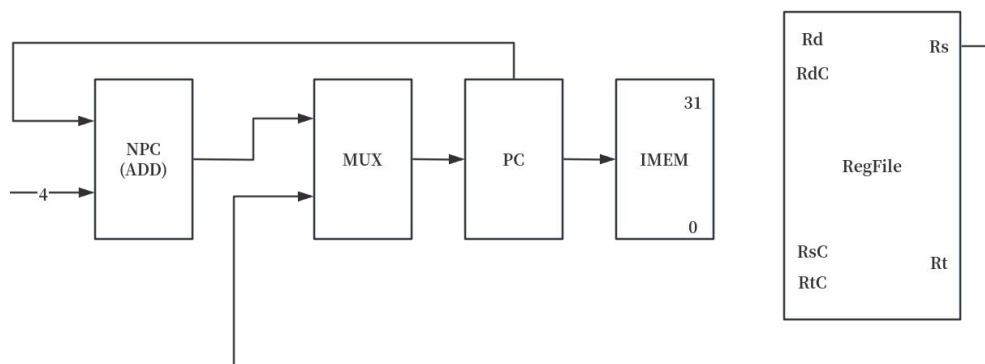
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs[4:0] -> EXT5
 EXT5 -> A
 Rt -> B
 (B >>> A -> Res)
 Res -> Rd

16 SRAV



PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 Rs[4:0] -> EXT5
 EXT5 -> A
 Rt -> B
 (B >> A -> Res)
 Res -> Rd

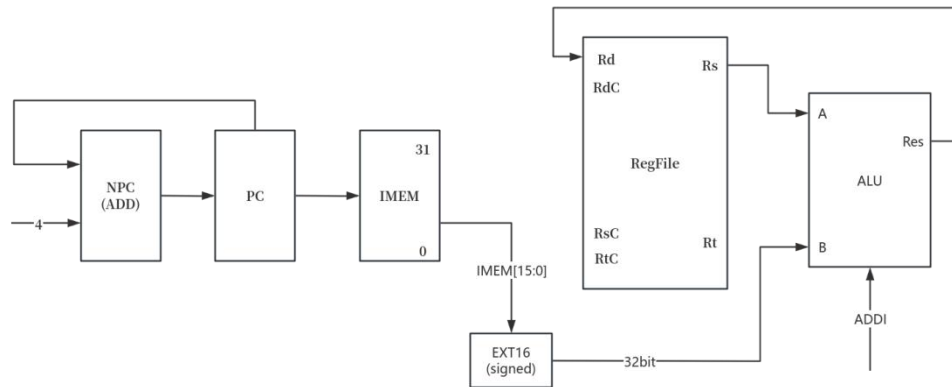
17 JR



PC -> IMEM
 PC + 4 -> NPC
 NPC -> MUX
 Rs -> MUX
 MUX -> PC

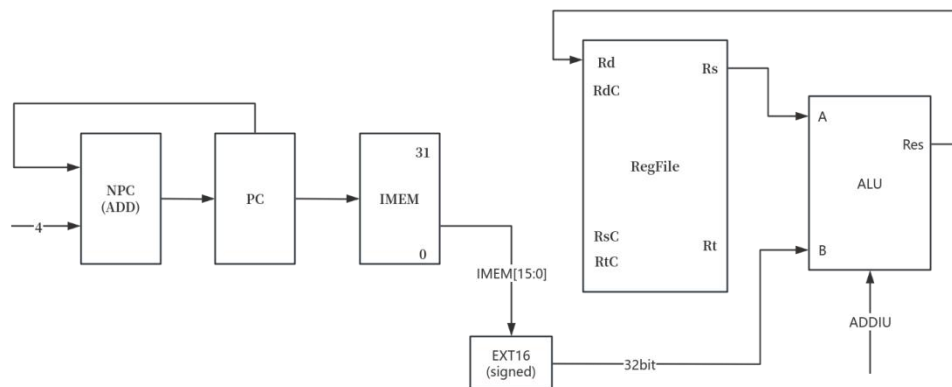
I 型指令

18 ADDI



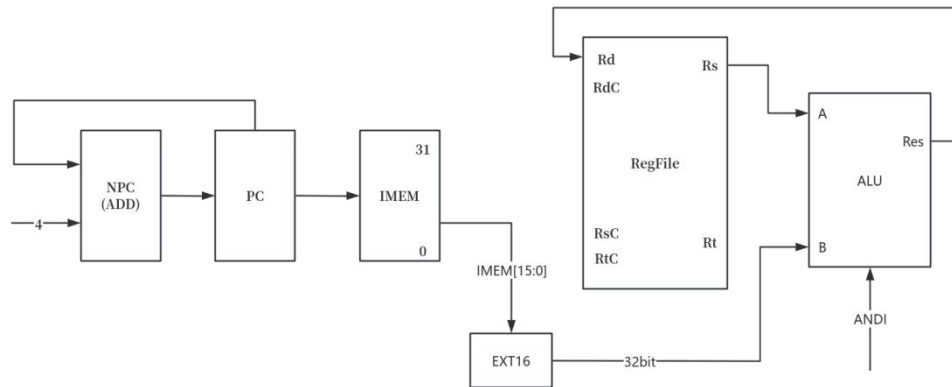
PC -> IMEM
PC + 4 -> NPC
NPC -> PC
IMEM[15:0] -> EXT16
Rs -> A
EXT16 -> B
(A + B -> Res)
Res -> Rd

19 ADDIU



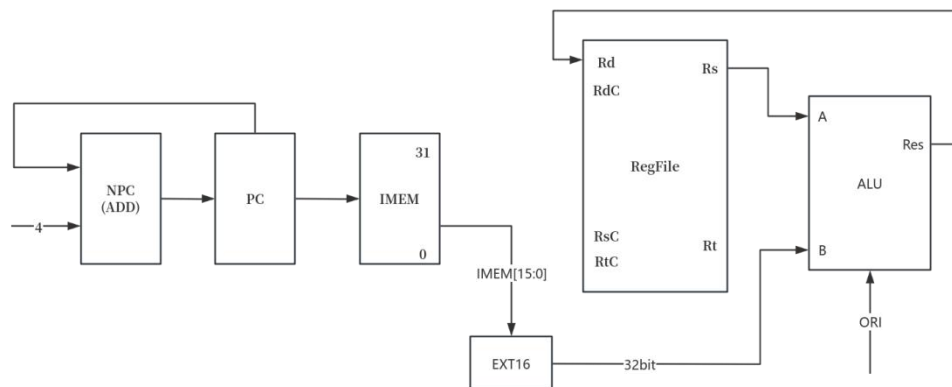
PC -> IMEM
PC + 4 -> NPC
NPC -> PC
IMEM[15:0] -> EXT16
Rs -> A
EXT16 -> B
(A + B -> Res)
Res -> Rd

20 ANDI



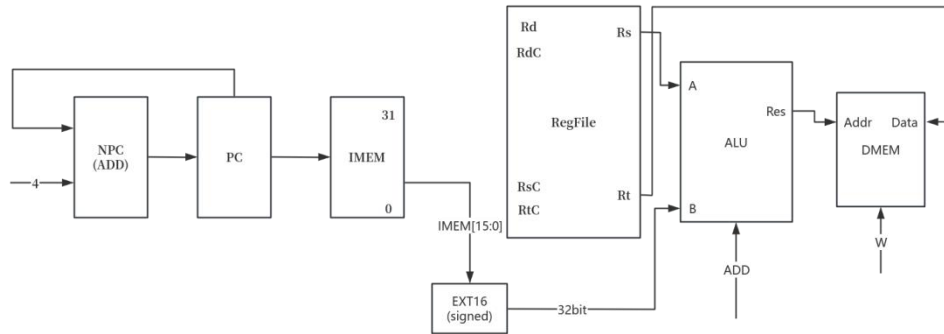
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 IMEM[15:0] -> EXT16
 Rs -> A
 EXT16 -> B
 (A & B -> Res)
 Res -> Rd

21 ORI



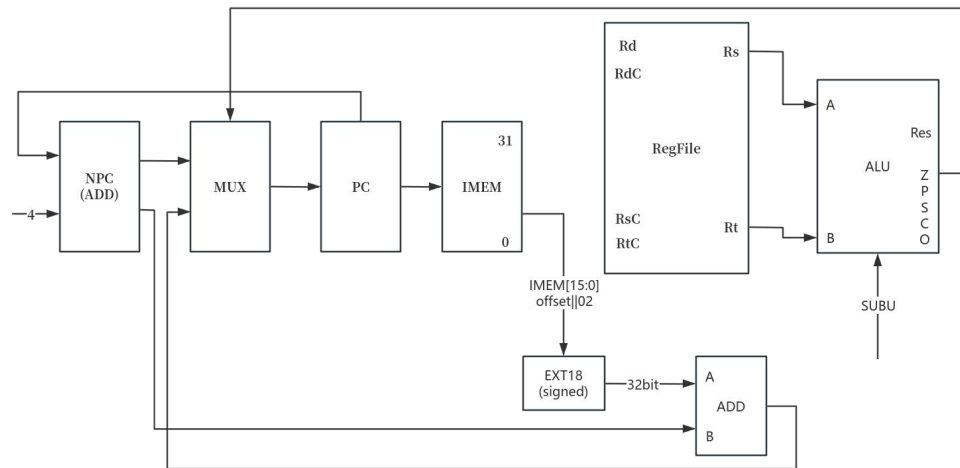
PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 IMEM[15:0] -> EXT16
 Rs -> A
 EXT16 -> B
 (A | B -> Res)
 Res -> Rd

22 XORI



PC -> IMEM
 PC + 4 -> NPC
 NPC -> PC
 IMEM[15:0] -> EXT16
 Rs -> A
 EXT16 -> B
 (A + B -> Res)
 Res -> DMEM_Addr
 Rt -> DMEM

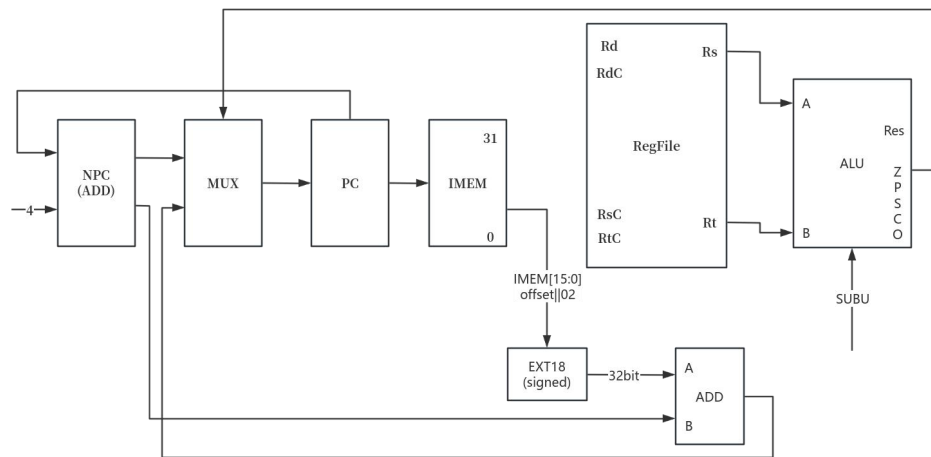
25 BEQ



PC -> IMEM
 PC + 4 -> NPC
 NPC -> MUX
 Rs -> A
 Rt -> B
 (A - B -> Res)
 ZF -> MUX
 IMEM[15:0]||02 -> EXT18
 EXT18 -> ADD_A
 NPC -> ADD_B
 (ADD_A + ADD_B -> ADD_OUT)

MUX -> PC

26 BNE



PC -> IMEM

PC + 4 -> NPC

NPC -> MUX

$$R_S \rightarrow A$$
$$R_t \rightarrow B$$
$$(A - B \rightarrow \text{Res})$$

ZF -> MUX

IMEM[15:0]||02 -> EXT18

EXT18 -> ADD_A

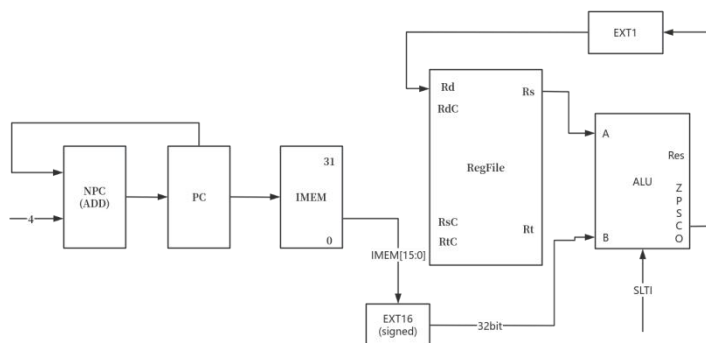
NPC -> ADD B

$$(\text{ADD_A} + \text{ADD_B} \rightarrow \text{ADD_OUT})$$

ADD -> MUX

MUX -> PC

27 SLTI



PC -> IMEM

PC + 4 -> NPC

NPC $\rightarrow$ PC

IMEM[15:0] -> EXT16

Rs -> A

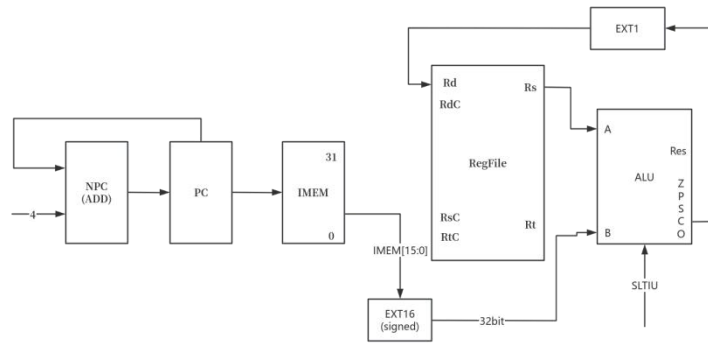
EXT16 -> B

(A - B -> Res)

CF -> EXT1

EXT1 -> Rd

28 SLTIU



PC -> IMEM

PC + 4 -> NPC

NPC -> PC

IMEM[15:0] -> EXT16

Rs -> A

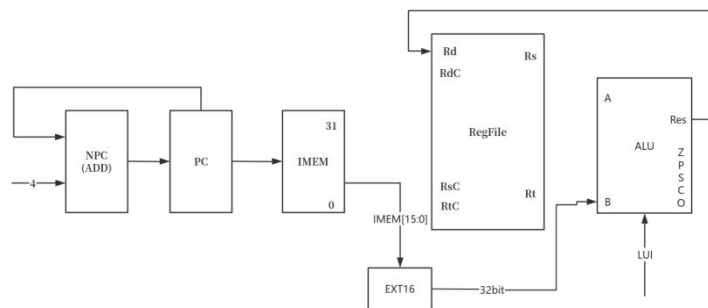
EXT16 -> B

(A - B -> Res)

CF -> EXT1

EXT1 -> Rd

29 LUI



PC -> IMEM

PC + 4 -> NPC

NPC -> PC

IMEM[15:0] -> EXT16

Rs -> A

EXT16 -> B

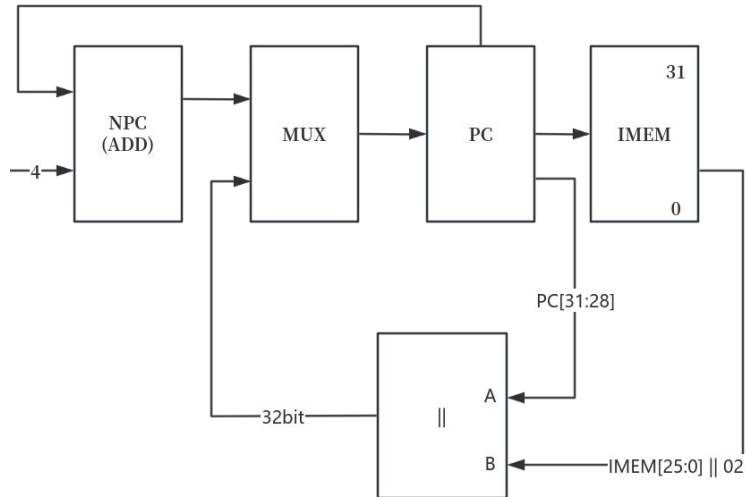
(A - B -> Res)

CF -> EXT1

EXT1 -> Rd

J 型指令

30 J



PC -> IMEM

PC + 4 -> NPC

NPC -> MUX

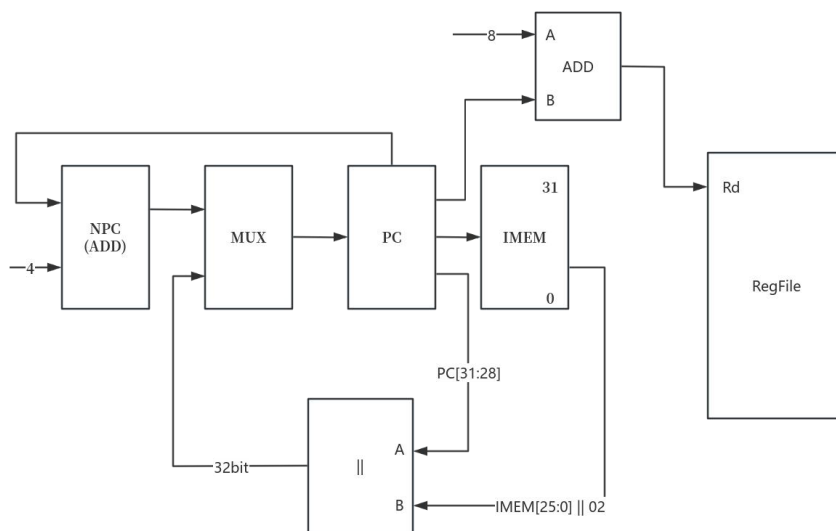
PC[31:28] -> ||_A

IMEM[25:0]||02 -> ||_B

|| -> MUX

MUX -> PC

31 JAL

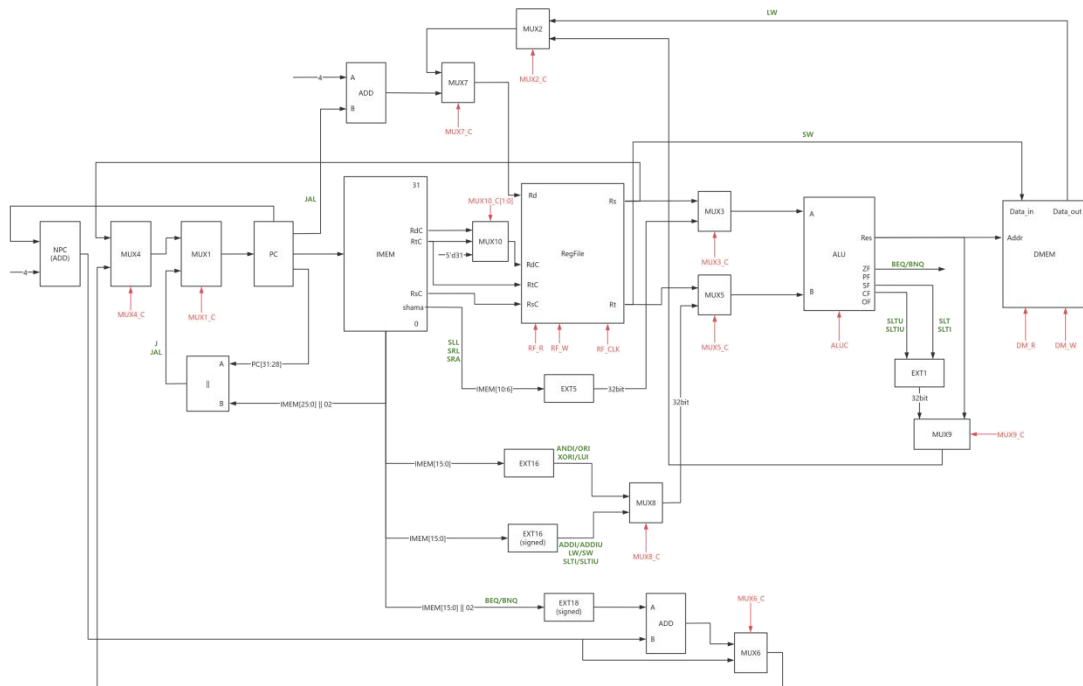


PC -> IMEM

PC + 4 -> NPC

NPC -> MUX
 8 -> ADD_A
 PC -> ADD_B
 (ADD_A + ADD_B -> ADD_OUT)
 ADD -> Rd
 PC[31:28] -> ||_A
 IMEM[25:0]||02 -> ||_B
 || -> MUX
 MUX -> PC

(3) 31 条指令总数据通路图。



三、CPU 控制部件设计

(1) 根据指令流程图，编排指令取指到执行的操作时间表。

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	指令	MUX1	MUX2	MUX3	MUX4	MUX5	MUX6	MUX7	MUX8	MUX9	MUX10	ALUC	DMEM		EXT1	RegFile
2												ALUC[3:0]	DM_R	DM_W		RF_W
3	R型指令															
4	ADD	0	1	0	1	0	1	0		1	00	ADD(0010)				1
5	ADDU	0	1	0	1	0	1	0		1	00	ADDU(0000)				1
6	SUB	0	1	0	1	0	1	0		1	00	SUB(0011)				1
7	SUBU	0	1	0	1	0	1	0		1	00	SUBU(0001)				1
8	AND	0	1	0	1	0	1	0		1	00	AND(0100)				1
9	OR	0	1	0	1	0	1	0		1	00	OR(0101)				1
10	XOR	0	1	0	1	0	1	0		1	00	XOR(0110)				1
11	NOR	0	1	0	1	0	1	0		1	00	NOR(0111)				1
12	SLT	0	1	0	1	0	1	0		0	00	SLT(1011)			ALU-SF	1
13	SLTU	0	1	0	1	0	1	0		0	00	SLTU(1010)			ALU-CF	1
14	SLL	0	1	1	1	0	1	0		1	00	SLL(111X)				1
15	SRL	0	1	1	1	0	1	0		1	00	SRL(1101)				1
16	SRA	0	1	1	1	0	1	0		1	00	SRA(1100)				1
17	SLLV	0	1	0	1	0	1	0		1	00	SLL(111X)				1
18	SRLV	0	1	0	1	0	1	0		1	00	SRL(1101)				1
19	SRAV	0	1	0	1	0	1	0		1	00	SRA(1100)				1
20	JR	0			0											0
21	I型指令															
22	ADDI	0	1	0	1	1	1	0	1	1	01	ADD(0010)				1
23	ADDIU	0	1	0	1	1	1	0	1	1	01	ADDU(0000)				1
24	ANDI	0	1	0	1	1	1	0	0	1	01	AND(0100)				1
25	ORI	0	1	0	1	1	1	0	0	1	01	OR(0101)				1
26	XORI	0	1	0	1	1	1	0	0	1	01	XOR(0110)				1
27	LW	0	0	0	1	1	1	0	1		01	ADDU(0000)	1	0		1
28	SW	0		0	1	1	1		1			ADDU(0000)	0	1		0
29	BEQ	0		0	1	0	ALU-ZF					SUBU(0001)				0
30	BNE	0		0	1	0	!ALU-ZF					SUBU(0001)				0
31	SLTI	0	1	0	1	1	1	0	1	0	01	SLT(1011)			ALU-SF	1
32	SLTIU	0	1	0	1	1	1	0	1	0	01	SLTU(1010)			ALU-CF	1
33	LUI	0	1		1	1	1	0	0	1	01	LUI(100X)				1
34	J型指令															
35	J	1														0
36	JAL	1						1			10					1

详细见附件-指令操作时间表.xlsx

(2) 根据指令操作时间表，写出每个控制信号的逻辑表达式。

以下是我 CPU 设计中控制器 CU 的所有控制信号：

1. 一共 10 个多路选择器 MUX 控制信号：muxc[8:0]和 mux10c[1:0]
2. 逻辑运算单元 ALU 控制信号：aluc[3:0]
- 3.1 位扩展器控制信号：EXT1_C
4. 数据存储器 DMEM 控制信号：写使能 DM_W，读使能 DM_R
5. 寄存器堆 RegFile 控制信号：写使能 RF_W

（其余的一些部件比如 16 位扩展器 EXT16、18 位扩展器 EXT18、拼接器 CAT 等等由于输入来源只有一个，不需要特定的控制信号，因此简化掉）

控制信号的逻辑表达式：（定义指令标志位，比如 ADD 指令代表 ADD_flag=1）

$\text{muxc8} = \text{ADD_flag} \parallel \text{ADDU_flag} \parallel \text{SUB_flag} \parallel \text{SUBU_flag} \parallel \text{AND_flag} \parallel \text{OR_flag} \parallel \text{XOR_flag} \parallel \text{NOR_flag} \parallel \text{SLL_flag} \parallel \text{SRL_flag} \parallel \text{SRA_flag} \parallel \text{SLLV_flag} \parallel \text{SRLV_flag} \parallel \text{SRAV_flag} \parallel \text{ADDI_flag} \parallel \text{ADDIU_flag} \parallel \text{ANDI_flag} \parallel \text{ORI_flag} \parallel \text{XORI_flag} \parallel \text{LUI_flag};$

$\text{muxc7} = \text{ADDI_flag} \parallel \text{ADDIU_flag} \parallel \text{SLTI_flag} \parallel \text{SLTIU_flag};$

$\text{muxc6} = \text{SLL_flag} \parallel \text{SRL_flag} \parallel \text{SRA_flag};$

$\text{muxc5} = \text{ADD_flag} \parallel \text{ADDU_flag} \parallel \text{SUB_flag} \parallel \text{SUBU_flag} \parallel \text{AND_flag} \parallel \text{OR_flag} \parallel \text{XOR_flag} \parallel \text{NOR_flag} \parallel \text{SLT_flag} \parallel \text{SLTU_flag} \parallel \text{SLL_flag} \parallel \text{SRL_flag} \parallel \text{SRA_flag} \parallel \text{SLLV_flag} \parallel \text{SRLV_flag} \parallel \text{SRAV_flag} \parallel \text{ADDI_flag} \parallel \text{ADDIU_flag} \parallel \text{ANDI_flag} \parallel \text{ORI_flag} \parallel \text{XORI_flag} \parallel \text{LW_flag} \parallel \text{SW_flag} \parallel \text{BEQ_TAKEN_flag} \parallel \text{BNE_TAKEN_flag} \parallel \text{SLTI_flag} \parallel \text{SLTIU_flag} \parallel \text{LUI_flag};$

$\text{muxc4} = \text{ADDI_flag} \parallel \text{ADDIU_flag} \parallel \text{ANDI_flag} \parallel \text{ORI_flag} \parallel \text{XORI_flag} \parallel \text{LW_flag} \parallel \text{SW_flag} \parallel \text{SLTI_flag} \parallel \text{SLTIU_flag} \parallel \text{LUI_flag};$

```

muxc3 = BEQ_flag || BNE_flag || ADD_flag || ADDU_flag || SUB_flag || SUBU_flag
        || AND_flag || OR_flag || XOR_flag || NOR_flag || SLT_flag || SLTU_flag ||
        SLL_flag || SRL_flag || SRA_flag || SLLV_flag || SRLV_flag || SRAV_flag ||
        ADDI_flag || ADDIU_flag || ANDI_flag || ORI_flag || XORI_flag || LW_flag
        || SW_flag || SLTI_flag || SLTIU_flag || LUI_flag;
muxc2 = JAL_flag;
muxc1 = ADDI_flag || ADDIU_flag || LW_flag || SW_flag || BEQ_flag || BNE_flag ||
        SLTI_flag || SLTIU_flag;
muxc0 = J_flag || JAL_flag;
mux10c1 = JAL_flag;
mux10c0 = ADDI_flag || ADDIU_flag || ANDI_flag || ORI_flag || XORI_flag ||
        LW_flag || SLTI_flag || SLTIU_flag || LUI_flag;
aluc3 = LUI_flag || SLL_flag || SRL_flag || SRA_flag || SLLV_flag || SRLV_flag ||
        SRAV_flag;
aluc2 = SLT_flag || SLTI_flag || SLTU_flag || SLTIU_flag || SLL_flag || SRL_flag ||
        SRA_flag || SLLV_flag || SRLV_flag || SRAV_flag;
aluc1 = ADD_flag || ADDI_flag || SUB_flag || BEQ_flag || BNE_flag || XOR_flag ||
        XORI_flag || NOR_flag || SLT_flag || SLTI_flag || SLTU_flag || SLTIU_flag
        || SLL_flag || SLLV_flag;
aluc0 = SUB_flag || SUBU_flag || BEQ_flag || BNE_flag || OR_flag || ORI_flag ||
        NOR_flag || SLT_flag || SLTI_flag || SRL_flag || SRLV_flag;
EXT1_C = SLTI_flag || SLTIU_flag;
DM_W = SW_flag;
DM_R = LW_flag;
RF_W = ADD_flag || ADDU_flag || SUB_flag || SUBU_flag || AND_flag || OR_flag ||
        XOR_flag || NOR_flag || SLT_flag || SLTU_flag || SLL_flag || SRL_flag ||
        SRA_flag || SLLV_flag || SRLV_flag || SRAV_flag ||
        ADDI_flag || ADDIU_flag || ANDI_flag || ORI_flag || XORI_flag ||
        LW_flag || SLTI_flag || SLTIU_flag || LUI_flag || JAL_flag;

```

(3) 根据逻辑表达式，完成控制部件 CU 的设计。

```

//控制器（内置译码器）
module CU(
    input [31:0] inst, // 指令
    input ZF, // 零标志位,MUX6 要用
    output [8:0] muxc, // 多路选择器控制
    output [1:0] mux10c, // 多路选择器控制 RF 的写入地址: 00-Rdc, 01-Rtc, 10-$ra
    output [3:0] aluc, // ALU 操作码
    output EXT1_C, // 扩展 1 控制
    output RF_W, // 寄存器写使能
    output DM_W, // 数据存储器写使能
    output DM_R // 数据存储器读使能
);

```

```

// 指令字段解码
wire [5:0] op = inst[31:26]; // 操作码字段
wire [5:0] func = inst[5:0]; // 功能码字段

// Opcode 参数定义
localparam R_TYPE_OP = 6'b000000; // R 型指令操作码
localparam ADDI_OP = 6'b001000; // ADDI 指令操作码
localparam ADDIU_OP = 6'b001001; // ADDIU 指令操作码
localparam ANDI_OP = 6'b001100; // ANDI 指令操作码
localparam ORI_OP = 6'b001101; // ORI 指令操作码
localparam XORI_OP = 6'b001110; // XORI 指令操作码
localparam LW_OP = 6'b100011; // LW 指令操作码
localparam SW_OP = 6'b101011; // SW 指令操作码
localparam BEQ_OP = 6'b000100; // BEQ 指令操作码
localparam BNE_OP = 6'b000101; // BNE 指令操作码
localparam SLTI_OP = 6'b001010; // SLTI 指令操作码
localparam SLTIU_OP = 6'b001011; // SLTIU 指令操作码
localparam LUI_OP = 6'b001111; // LUI 指令操作码
localparam J_OP = 6'b000010; // J 指令操作码
localparam JAL_OP = 6'b000011; // JAL 指令操作码

// R 型指令功能码参数定义
localparam ADD_FUNC = 6'b100000; // ADD 功能码
localparam ADDU_FUNC = 6'b100001; // ADDU 功能码
localparam SUB_FUNC = 6'b100010; // SUB 功能码
localparam SUBU_FUNC = 6'b100011; // SUBU 功能码
localparam AND_FUNC = 6'b100100; // AND 功能码
localparam OR_FUNC = 6'b100101; // OR 功能码
localparam XOR_FUNC = 6'b100110; // XOR 功能码
localparam NOR_FUNC = 6'b100111; // NOR 功能码
localparam SLT_FUNC = 6'b101010; // SLT 功能码
localparam SLTU_FUNC = 6'b101011; // SLTU 功能码
localparam SLL_FUNC = 6'b000000; // SLL 功能码
localparam SRL_FUNC = 6'b000010; // SRL 功能码
localparam SRA_FUNC = 6'b000011; // SRA 功能码
localparam SLLV_FUNC = 6'b000100; // SLLV 功能码
localparam SRLV_FUNC = 6'b000110; // SRLV 功能码
localparam SRAV_FUNC = 6'b000111; // SRAV 功能码
localparam JR_FUNC = 6'b001000; // JR 功能码

// ALUC
localparam ALU_ADDU = 4'b0000; // 无符号加
localparam ALU_SUBU = 4'b0001; // 无符号减
localparam ALU_ADD = 4'b0010; // 有符号加
localparam ALU_SUB = 4'b0011; // 有符号减
localparam ALU_AND = 4'b0100; // 逻辑与
localparam ALU_OR = 4'b0101; // 逻辑或

```

```

localparam ALU_XOR  = 4'b0110; // 逻辑异或
localparam ALU_NOR  = 4'b0111; // 逻辑或非
localparam ALU_LUI  = 4'b1000; // LUI: 立即数加载到高位
localparam ALU_SLTU = 4'b1010; // 无符号小于则置位
localparam ALU_SLT  = 4'b1011; // 有符号小于则置位
localparam ALU_SRA  = 4'b1100; // 算术右移
localparam ALU_SRL  = 4'b1101; // 逻辑右移
localparam ALU_SLL  = 4'b1110; // 逻辑左移

// R 型指令标志位
wire ADD_flag    = (op == R_TYPE_OP && func == ADD_FUNC);
wire ADDU_flag   = (op == R_TYPE_OP && func == ADDU_FUNC);
wire SUB_flag    = (op == R_TYPE_OP && func == SUB_FUNC);
wire SUBU_flag   = (op == R_TYPE_OP && func == SUBU_FUNC);
wire AND_flag    = (op == R_TYPE_OP && func == AND_FUNC);
wire OR_flag     = (op == R_TYPE_OP && func == OR_FUNC);
wire XOR_flag    = (op == R_TYPE_OP && func == XOR_FUNC);
wire NOR_flag    = (op == R_TYPE_OP && func == NOR_FUNC);
wire SLT_flag    = (op == R_TYPE_OP && func == SLT_FUNC);
wire SLTU_flag   = (op == R_TYPE_OP && func == SLTU_FUNC);
wire SLL_flag    = (op == R_TYPE_OP && func == SLL_FUNC);
wire SRL_flag    = (op == R_TYPE_OP && func == SRL_FUNC);
wire SRA_flag    = (op == R_TYPE_OP && func == SRA_FUNC);
wire SLLV_flag   = (op == R_TYPE_OP && func == SLLV_FUNC);
wire SRLV_flag   = (op == R_TYPE_OP && func == SRLV_FUNC);
wire SRAV_flag   = (op == R_TYPE_OP && func == SRAV_FUNC);
wire JR_flag     = (op == R_TYPE_OP && func == JR_FUNC);

// I 型指令标志位
wire ADDI_flag   = (op == ADDI_OP);
wire ADDIU_flag  = (op == ADDIU_OP);
wire ANDI_flag   = (op == ANDI_OP);
wire ORI_flag    = (op == ORI_OP);
wire XORI_flag   = (op == XORI_OP);
wire LW_flag     = (op == LW_OP);
wire SW_flag     = (op == SW_OP);
wire BEQ_flag    = (op == BEQ_OP);
wire BNE_flag    = (op == BNE_OP);
wire SLTI_flag   = (op == SLTI_OP);
wire SLTIU_flag  = (op == SLTIU_OP);
wire LUI_flag    = (op == LUI_OP);

// J 型指令标志位
wire J_flag      = (op == J_OP);
wire JAL_flag    = (op == JAL_OP);

// 分支条件标志位
wire BEQ_TAKEN_flag = BEQ_flag && !ZF;

```

```

wire BNE_TAKEN_flag = BNE_flag && ZF;
// ALU 控制信号生成
// aluc[3:0] = {aluc3, aluc2, aluc1, aluc0}
wire aluc3 = LUI_flag || SLL_flag || SRL_flag || SRA_flag || SLLV_flag || SRLV_flag || SRAV_flag;
wire aluc2 = SLT_flag || SLTI_flag || SLTU_flag || SLTIU_flag || SLL_flag || SRL_flag || SRA_flag
||
        SLLV_flag || SRLV_flag || SRAV_flag;
wire aluc1 = ADD_flag || ADDI_flag || SUB_flag || BEQ_flag || BNE_flag || XOR_flag || XORI_flag
||
        NOR_flag || SLT_flag || SLTI_flag || SLTU_flag || SLTIU_flag || SLL_flag || SLLV_flag;
wire aluc0 = SUB_flag || SUBU_flag || BEQ_flag || BNE_flag || OR_flag || ORI_flag || NOR_flag ||
        SLT_flag || SLTI_flag || SRL_flag || SRLV_flag;
// 寄存器写使能
wire RF_W_wire = ADD_flag || ADDU_flag || SUB_flag || SUBU_flag || AND_flag || OR_flag ||
        XOR_flag || NOR_flag || SLT_flag || SLTU_flag || SLL_flag || SRL_flag ||
        SRA_flag || SLLV_flag || SRLV_flag || SRAV_flag ||
        ADDI_flag || ADDIU_flag || ANDI_flag || ORI_flag || XORI_flag ||
        LW_flag || SLTI_flag || SLTIU_flag || LUI_flag || JAL_flag;
// 数据存储器控制信号
wire DM_W_wire = SW_flag;
wire DM_R_wire = LW_flag;
// 多路复用器控制信号
// muxc[8:0] = {muxc8, muxc7, muxc6, muxc5, muxc4, muxc3, muxc2, muxc1, muxc0}
wire muxc8 = ADD_flag || ADDU_flag || SUB_flag || SUBU_flag || AND_flag || OR_flag ||
        XOR_flag || NOR_flag || SLL_flag || SRL_flag || SRA_flag || SLLV_flag ||
        SRLV_flag || SRAV_flag || ADDI_flag || ADDIU_flag || ANDI_flag ||
        ORI_flag || XORI_flag || LUI_flag;
wire muxc7 = ADDI_flag || ADDIU_flag || SLTI_flag || SLTIU_flag;
wire muxc6 = SLL_flag || SRL_flag || SRA_flag;
wire muxc5 = ADD_flag || ADDU_flag || SUB_flag || SUBU_flag || AND_flag || OR_flag ||
        XOR_flag || NOR_flag || SLT_flag || SLTU_flag || SLL_flag || SRL_flag ||
        SRA_flag || SLLV_flag || SRLV_flag || SRAV_flag || ADDI_flag || ADDIU_flag ||
        ANDI_flag || ORI_flag || XORI_flag || LW_flag || SW_flag || BEQ_TAKEN_flag ||
        BNE_TAKEN_flag || SLTI_flag || SLTIU_flag || LUI_flag;
wire muxc4 = ADDI_flag || ADDIU_flag || ANDI_flag || ORI_flag || XORI_flag ||
        LW_flag || SW_flag || SLTI_flag || SLTIU_flag || LUI_flag;
wire muxc3 = BEQ_flag || BNE_flag || ADD_flag || ADDU_flag || SUB_flag ||
        SUBU_flag || AND_flag || OR_flag || XOR_flag || NOR_flag || SLT_flag ||
        SLTU_flag || SLL_flag || SRL_flag || SRA_flag || SLLV_flag || SRLV_flag ||
        SRAV_flag || ADDI_flag || ADDIU_flag || ANDI_flag || ORI_flag || XORI_flag ||
        LW_flag || SW_flag || SLTI_flag || SLTIU_flag || LUI_flag;
wire muxc2 = JAL_flag;
wire muxc1 = ADDI_flag || ADDIU_flag || LW_flag || SW_flag || BEQ_flag || BNE_flag ||
        SLTI_flag || SLTIU_flag;

```

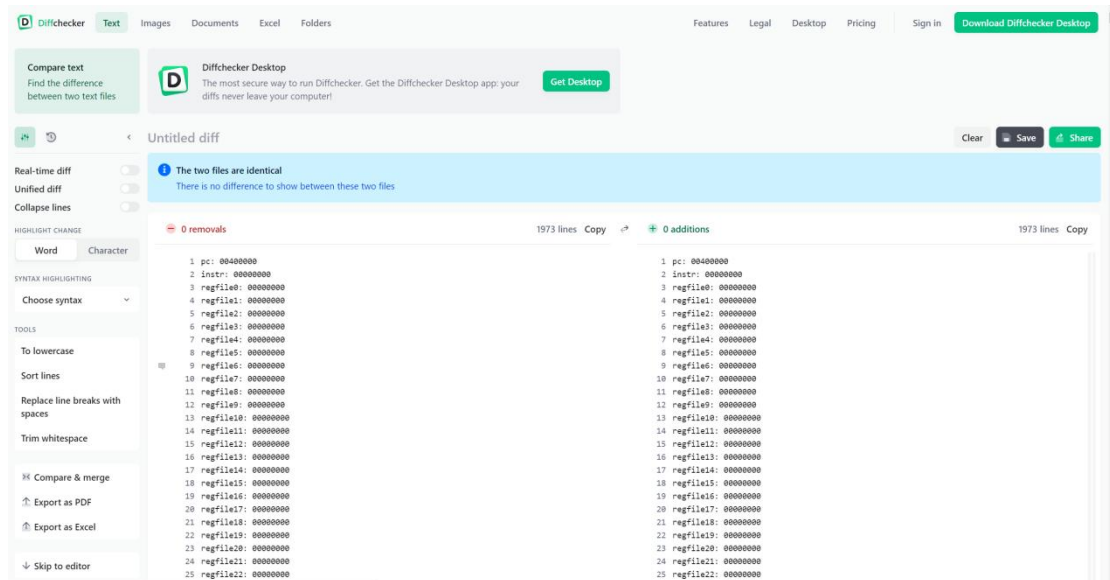

(2) 对单条指令、网站前仿真所用测试代码 `coe` 文件等测试的结果进行比对。

用 Mars 运行 MIPS 汇编程序后会在 Mars 目录下生成一个 `result.txt` 文件

用自己的 CPU 运行导出的 MIPS 汇编程序，将 `pc`、`instr`、`regfile` 结果输出到文件，在 CPU 工程目录下生成另一个 `result.txt` 文件。

将两个文件进行比对。下面放了一些指令的对比结果，不论是寄存器堆中的值，还是数据存储器中的值，都和 Mars 运行的结果相同，可以验证设计的 CPU 是没问题的。

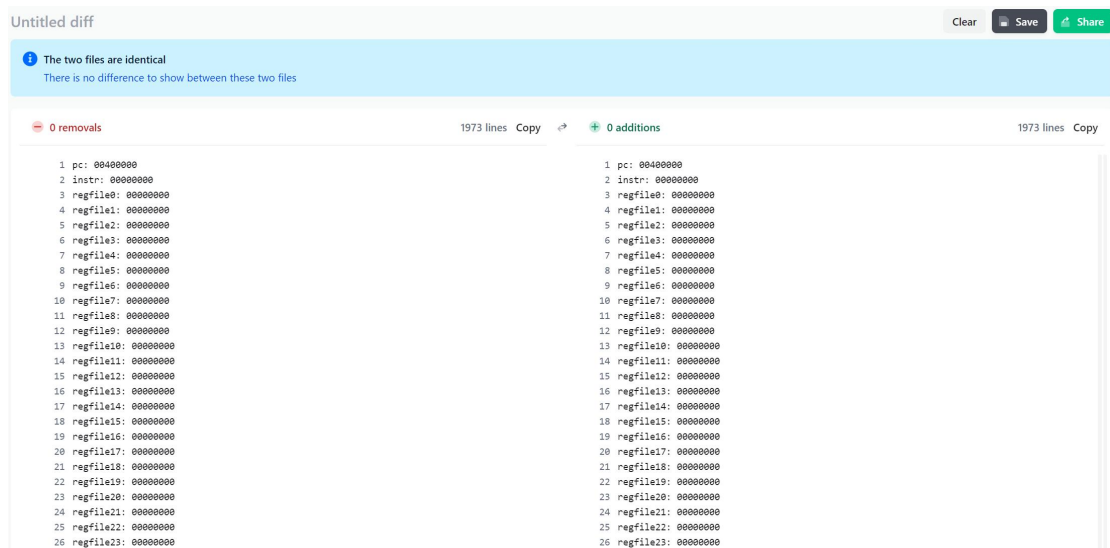
addi



The screenshot shows the Diffchecker website interface. At the top, there's a navigation bar with links like 'Features', 'Legal', 'Desktop', 'Pricing', 'Sign in', and a 'Download Diffchecker Desktop' button. Below the navigation bar, there's a 'Compare text' section with a 'Find the difference between two text files' input. To the right of this, there's a 'Diffchecker Desktop' section with a 'Get Desktop' button. The main area shows a comparison of two files, both of which are identical. The files contain MIPS assembly code for the 'addi' instruction. The code is as follows:

```
1 pc: 00400000
2 instr: 00000000
3 regfile0: 00000000
4 regfile1: 00000000
5 regfile2: 00000000
6 regfile3: 00000000
7 regfile4: 00000000
8 regfile5: 00000000
9 regfile6: 00000000
10 regfile7: 00000000
11 regfile8: 00000000
12 regfile9: 00000000
13 regfile10: 00000000
14 regfile11: 00000000
15 regfile12: 00000000
16 regfile13: 00000000
17 regfile14: 00000000
18 regfile15: 00000000
19 regfile16: 00000000
20 regfile17: 00000000
21 regfile18: 00000000
22 regfile19: 00000000
23 regfile20: 00000000
24 regfile21: 00000000
25 regfile22: 00000000
```

addiu



The screenshot shows the Diffchecker website interface. At the top, there's a navigation bar with links like 'Features', 'Legal', 'Desktop', 'Pricing', 'Sign in', and a 'Download Diffchecker Desktop' button. Below the navigation bar, there's a 'Compare text' section with a 'Find the difference between two text files' input. To the right of this, there's a 'Diffchecker Desktop' section with a 'Get Desktop' button. The main area shows a comparison of two files, both of which are identical. The files contain MIPS assembly code for the 'addiu' instruction. The code is as follows:

```
1 pc: 00400000
2 instr: 00000000
3 regfile0: 00000000
4 regfile1: 00000000
5 regfile2: 00000000
6 regfile3: 00000000
7 regfile4: 00000000
8 regfile5: 00000000
9 regfile6: 00000000
10 regfile7: 00000000
11 regfile8: 00000000
12 regfile9: 00000000
13 regfile10: 00000000
14 regfile11: 00000000
15 regfile12: 00000000
16 regfile13: 00000000
17 regfile14: 00000000
18 regfile15: 00000000
19 regfile16: 00000000
20 regfile17: 00000000
21 regfile18: 00000000
22 regfile19: 00000000
23 regfile20: 00000000
24 regfile21: 00000000
25 regfile22: 00000000
26 regfile23: 00000000
```

lui

Clear Save Share

0 removals 1123 lines Copy 0 additions 1123 lines Copy

0 removals 1123 lines Copy ↔ 0 additions 1123 lines Copy

1wsw

Clear Save Share Explain

3468 removals 6801 lines Copy 0 additions 3333 lines Copy

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0xffffffff	0x000000ff	0x00000fff	0x0000ffff	0xffffffff	0x00005555	0xffffffff
0x10010020	0xffffbbbb	0xfffecccc	0xffffddd4	0xffffeeee	0xffffffff	0xfffd0000	0x000f0000	0x00ff0000
0x10010040	0x0ffff000	0xffef0000	0xf5550000	0xaabbbb0000	0xccee+e000	0xdddd0000	0xbbeee0000	0x00000000
0x10010060	0xffff0000	0xfffF0000	0x00FF0000	0x00FF0000	0xffff0000	0xfffd0000	0x55550000	0x000000ff
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

< |
 Oxt10010000 (.data) ▾ |
☒ Hexadecimal Addresses
 ☒ Hexadecimal Values
 ☐ ASCII

mem [31:0] [3...	0000000f...	Array
[31] [31:0]	0000000f	Array
[30] [31:0]	55550000	Array
[29] [31:0]	ffff0000	Array
[28] [31:0]	0fff0000	Array
[27] [31:0]	00ff0000	Array
[26] [31:0]	000f0000	Array
[25] [31:0]	ffff0000	Array
[24] [31:0]	ffff0000	Array
[23] [31:0]	eeee0000	Array
[22] [31:0]	dddd0000	Array
[21] [31:0]	cccc0000	Array
[20] [31:0]	bbbb0000	Array
[19] [31:0]	aaaa0000	Array
[18] [31:0]	55550000	Array
[17] [31:0]	ffff0000	Array
[16] [31:0]	0fff0000	Array
[15] [31:0]	00ff0000	Array
[14] [31:0]	000f0000	Array
[13] [31:0]	ffff0000	Array
[12] [31:0]	fffffff	Array
[11] [31:0]	ffffeee	Array
[10] [31:0]	ffffddd	Array
[9] [31:0]	ffffccc	Array
[8] [31:0]	ffffbbb	Array
[7] [31:0]	ffffaaa	Array
[6] [31:0]	0005555	Array
[5] [31:0]	fffffff	Array
[4] [31:0]	0000fff	Array
[3] [31:0]	00000ff	Array
[2] [31:0]	000000f	Array
[1] [31:0]	fffffff	Array
[0] [31:0]	0000000	Array

lws2

Untitled diff

Clear

Save

Share

Explain

3468 removals

6801 lines Copy

0 additions

3333 lines Copy

1 pc: 00400000

2 instr: 00000000

3 regfile0: 00000000

4 regfile1: 00000000

5 regfile2: 00000000

6 regfile3: 00000000

7 regfile4: 00000000

8 regfile5: 00000000

9 regfile6: 00000000

10 regfile7: 00000000

11 regfile8: 00000000

12 regfile9: 00000000

13 regfile10: 00000000

14 regfile11: 00000000

15 regfile12: 00000000

16 regfile13: 00000000

17 regfile14: 00000000

18 regfile15: 00000000

19 regfile16: 00000000

20 regfile17: 00000000

21 regfile18: 00000000

22 regfile19: 00000000

23 regfile20: 00000000

24 regfile21: 00000000

25 regfile22: 00000000

26 regfile23: 00000000

27 regfile24: 00000000

28 regfile25: 00000000

29 regfile26: 00000000

30 regfile27: 00000000

1 pc: 00400000

2 instr: 00000000

3 regfile0: 00000000

4 regfile1: 00000000

5 regfile2: 00000000

6 regfile3: 00000000

7 regfile4: 00000000

8 regfile5: 00000000

9 regfile6: 00000000

10 regfile7: 00000000

11 regfile8: 00000000

12 regfile9: 00000000

13 regfile10: 00000000

14 regfile11: 00000000

15 regfile12: 00000000

16 regfile13: 00000000

17 regfile14: 00000000

18 regfile15: 00000000

19 regfile16: 00000000

20 regfile17: 00000000

21 regfile18: 00000000

22 regfile19: 00000000

23 regfile20: 00000000

24 regfile21: 00000000

25 regfile22: 00000000

26 regfile23: 00000000

27 regfile24: 00000000

28 regfile25: 00000000

29 regfile26: 00000000

30 regfile27: 00000000

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x00000006	0x00000007
0x10010020	0x00000008	0x00000009	0x0000000a	0x0000000b	0x0000000c	0x0000000d	0x0000000e	0x0000000f
0x10010040	0x00000010	0x00000011	0x00000012	0x00000013	0x00000014	0x00000015	0x00000016	0x00000017
0x10010060	0x00000018	0x00000019	0x0000001a	0x0000001b	0x0000001c	0x0000001d	0x0000001e	0x0000001f
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (.data)

Hexadecimal Addresses

Hexadecimal Values

ASCII

mem[31:0][31:0]	0000001f...	Array
[31][31:0]	0000001f	Array
[30][31:0]	0000001e	Array
[29][31:0]	0000001d	Array
[28][31:0]	0000001c	Array
[27][31:0]	0000001b	Array
[26][31:0]	0000001a	Array
[25][31:0]	00000019	Array
[24][31:0]	00000018	Array
[23][31:0]	00000017	Array
[22][31:0]	00000016	Array
[21][31:0]	00000015	Array
[20][31:0]	00000014	Array
[19][31:0]	00000013	Array
[18][31:0]	00000012	Array
[17][31:0]	00000011	Array
[16][31:0]	00000010	Array
[15][31:0]	0000000f	Array
[14][31:0]	0000000e	Array
[13][31:0]	0000000d	Array
[12][31:0]	0000000c	Array
[11][31:0]	0000000b	Array
[10][31:0]	0000000a	Array
[9][31:0]	00000009	Array
[8][31:0]	00000008	Array
[7][31:0]	00000007	Array
[6][31:0]	00000006	Array
[5][31:0]	00000005	Array
[4][31:0]	00000004	Array
[3][31:0]	00000003	Array
[2][31:0]	00000002	Array
[1][31:0]	00000001	Array
[0][31:0]	00000000	Array

srav

Untitled diff		Clear	Save	Share	Explain
4828 removals		6801 lines	Copy	0 additions	1973 lines
1 pc: 00400000	1 pc: 00400000				
2 instr: 00000000	2 instr: 00000000				
3 regfile0: 00000000	3 regfile0: 00000000				
4 regfile1: 00000000	4 regfile1: 00000000				
5 regfile2: 00000000	5 regfile2: 00000000				
6 regfile3: 00000000	6 regfile3: 00000000				
7 regfile4: 00000000	7 regfile4: 00000000				
8 regfile5: 00000000	8 regfile5: 00000000				
9 regfile6: 00000000	9 regfile6: 00000000				
10 regfile7: 00000000	10 regfile7: 00000000				
11 regfile8: 00000000	11 regfile8: 00000000				
12 regfile9: 00000000	12 regfile9: 00000000				
13 regfile10: 00000000	13 regfile10: 00000000				
14 regfile11: 00000000	14 regfile11: 00000000				
15 regfile12: 00000000	15 regfile12: 00000000				
16 regfile13: 00000000	16 regfile13: 00000000				
17 regfile14: 00000000	17 regfile14: 00000000				
18 regfile15: 00000000	18 regfile15: 00000000				
19 regfile16: 00000000	19 regfile16: 00000000				
20 regfile17: 00000000	20 regfile17: 00000000				
21 regfile18: 00000000	21 regfile18: 00000000				
22 regfile19: 00000000	22 regfile19: 00000000				
23 regfile20: 00000000	23 regfile20: 00000000				
24 regfile21: 00000000	24 regfile21: 00000000				
25 regfile22: 00000000	25 regfile22: 00000000				
26 regfile23: 00000000	26 regfile23: 00000000				
27 regfile24: 00000000	27 regfile24: 00000000				
28 regfile25: 00000000	28 regfile25: 00000000				
29 regfile26: 00000000	29 regfile26: 00000000				
30 regfile27: 00000000	30 regfile27: 00000000				

beq

Untitled diff

ClearSaveShareExplain

1054 removals3061 lines Copy0 additions2007 lines Copy

1 pc: 00400000

2 instr: 00000000

3 regfile0: 00000000

4 regfile1: 00000000

5 regfile2: 00000000

6 regfile3: 00000000

7 regfile4: 00000000

8 regfile5: 00000000

9 regfile6: 00000000

10 regfile7: 00000000

11 regfile8: 00000000

12 regfile9: 00000000

13 regfile10: 00000000

14 regfile11: 00000000

15 regfile12: 00000000

16 regfile13: 00000000

17 regfile14: 00000000

18 regfile15: 00000000

19 regfile16: 00000000

20 regfile17: 00000000

21 regfile18: 00000000

22 regfile19: 00000000

23 regfile20: 00000000

24 regfile21: 00000000

25 regfile22: 00000000

26 regfile23: 00000000

27 regfile24: 00000000

28 regfile25: 00000000

29 regfile26: 00000000

30 regfile27: 00000000

...

1 pc: 00400000

2 instr: 00000000

3 regfile0: 00000000

4 regfile1: 00000000

5 regfile2: 00000000

6 regfile3: 00000000

7 regfile4: 00000000

8 regfile5: 00000000

9 regfile6: 00000000

10 regfile7: 00000000

11 regfile8: 00000000

12 regfile9: 00000000

13 regfile10: 00000000

14 regfile11: 00000000

15 regfile12: 00000000

16 regfile13: 00000000

17 regfile14: 00000000

18 regfile15: 00000000

19 regfile16: 00000000

20 regfile17: 00000000

21 regfile18: 00000000

22 regfile19: 00000000

23 regfile20: 00000000

24 regfile21: 00000000

25 regfile22: 00000000

26 regfile23: 00000000

27 regfile24: 00000000

28 regfile25: 00000000

29 regfile26: 00000000

30 regfile27: 00000000

...

bne

Untitled diff

ClearSaveShareExplain

4250 removals6801 lines Copy0 additions2551 lines Copy

1 pc: 00400000

2 instr: 00000000

3 regfile0: 00000000

4 regfile1: 00000000

5 regfile2: 00000000

6 regfile3: 00000000

7 regfile4: 00000000

8 regfile5: 00000000

9 regfile6: 00000000

10 regfile7: 00000000

11 regfile8: 00000000

12 regfile9: 00000000

13 regfile10: 00000000

14 regfile11: 00000000

15 regfile12: 00000000

16 regfile13: 00000000

17 regfile14: 00000000

18 regfile15: 00000000

19 regfile16: 00000000

20 regfile17: 00000000

21 regfile18: 00000000

22 regfile19: 00000000

23 regfile20: 00000000

24 regfile21: 00000000

25 regfile22: 00000000

26 regfile23: 00000000

27 regfile24: 00000000

28 regfile25: 00000000

29 regfile26: 00000000

30 regfile27: 00000000

...

1 pc: 00400000

2 instr: 00000000

3 regfile0: 00000000

4 regfile1: 00000000

5 regfile2: 00000000

6 regfile3: 00000000

7 regfile4: 00000000

8 regfile5: 00000000

9 regfile6: 00000000

10 regfile7: 00000000

11 regfile8: 00000000

12 regfile9: 00000000

13 regfile10: 00000000

14 regfile11: 00000000

15 regfile12: 00000000

16 regfile13: 00000000

17 regfile14: 00000000

18 regfile15: 00000000

19 regfile16: 00000000

20 regfile17: 00000000

21 regfile18: 00000000

22 regfile19: 00000000

23 regfile20: 00000000

24 regfile21: 00000000

25 regfile22: 00000000

26 regfile23: 00000000

27 regfile24: 00000000

28 regfile25: 00000000

29 regfile26: 00000000

30 regfile27: 00000000

...

五、 CPU 后仿真测试结果

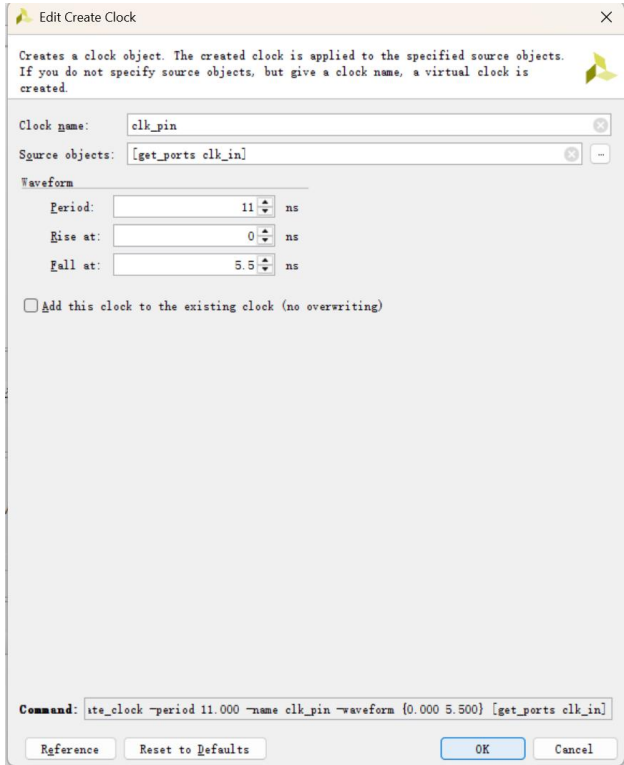
(1) 对 CPU 后仿真的实验内容和结果进行描述，并给出 modelsim 后仿真波形图的贴图。

后仿真也称为时序仿真或者布局布线后仿真，是指电路已经映射到特定的工艺环境以后，综合考虑电路的路径延迟与门延迟的影响，验证电路能否在一定时序条件下满足设计构想的过程，是否存在时序违规。其输入文件为从布局布线结果中抽象出来的门级网表、Testbench 和扩展名为 SDO 或 SDF 的标准时延文件。SDO 或 SDF 的标准时延文件不仅包含门延迟，还包括实际布线延迟，能较好地反映芯片的实际工作情况。一般来说后仿真是必选的，检查设计时序与实

际的 FPGA 运行情况是否一致，确保设计的可靠性和稳定性。选定了器件分配引脚后再做后仿真。

添加时钟约束

1.设置 create clock



The 'Edit Create Clock' dialog box is used to define a new clock. It includes a title bar with a yellow arrow icon and a close button. The main area contains a text box for 'Clock name' (set to 'clk_pin') and a list box for 'Source objects' (set to '[get_ports clk_in]'). Below these is a 'Waveform' section with three spinners for 'Period' (11 ns), 'Rise at' (0 ns), and 'Fall at' (5.5 ns). A checkbox 'Add this clock to the existing clock (no overwriting)' is present. At the bottom, a 'Command' text box shows the command: 'ste_clock -period 11.000 -name clk_pin -waveform {0.000 5.500} [get_ports clk_in]'. Buttons for 'Reference', 'Reset to Defaults', 'OK', and 'Cancel' are at the bottom.

Creates a clock object. The created clock is applied to the specified source objects. If you do not specify source objects, but give a clock name, a virtual clock is created.

Clock name:

Source objects:

Waveform

Period: ns

Rise at: ns

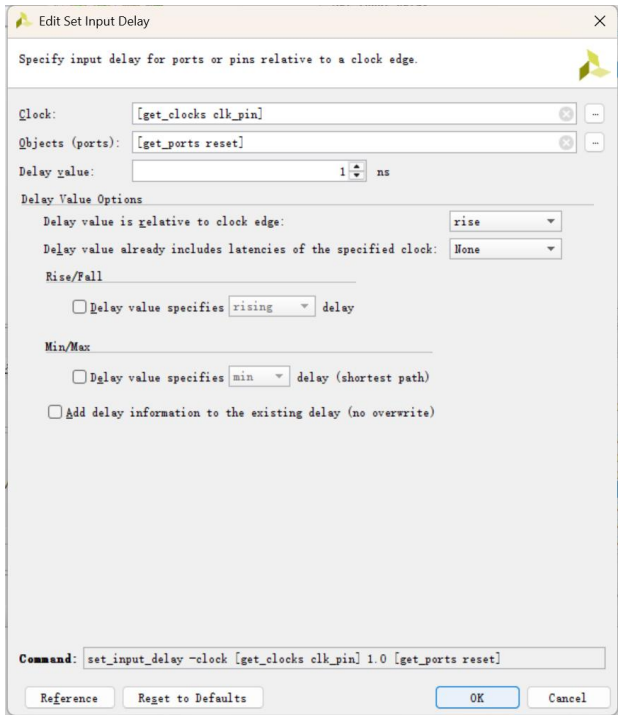
Fall at: ns

☐ Add this clock to the existing clock (no overwriting)

Command:

Reference Reset to Defaults OK Cancel

2.设置 Input Delay



The 'Edit Set Input Delay' dialog box is used to specify input delay for ports or pins relative to a clock edge. It includes a title bar with a yellow arrow icon and a close button. The main area contains a 'Clock' text box (set to '[get_clocks clk_pin]') and an 'Objects (ports)' list box (set to '[get_ports reset]'). A 'Delay value' spinner is set to 1 ns. Below is a 'Delay Value Options' section with three sub-sections: 'Delay value is relative to clock edge' (set to 'rise'), 'Delay value already includes latencies of the specified clock' (set to 'None'), and 'Rise/Fall' (set to 'rising'). There are checkboxes for 'Delay value specifies rising delay', 'Delay value specifies min delay (shortest path)', and 'Add delay information to the existing delay (no overwrite)'. At the bottom, a 'Command' text box shows the command: 'set_input_delay -clock [get_clocks clk_pin] 1.0 [get_ports reset]'. Buttons for 'Reference', 'Reset to Defaults', 'OK', and 'Cancel' are at the bottom.

Specify input delay for ports or pins relative to a clock edge.

Clock:

Objects (ports):

Delay value: ns

Delay Value Options

Delay value is relative to clock edge:

Delay value already includes latencies of the specified clock:

Rise/Fall

☐ Delay value specifies delay

Min/Max

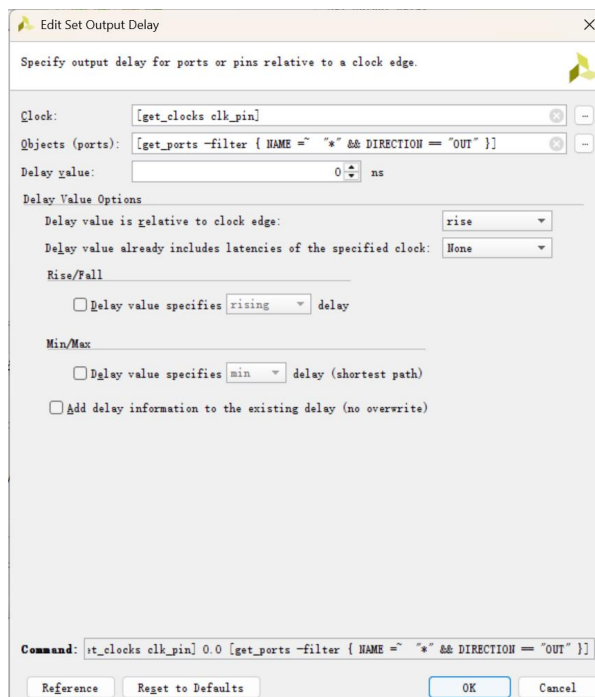
☐ Delay value specifies delay (shortest path)

☐ Add delay information to the existing delay (no overwrite)

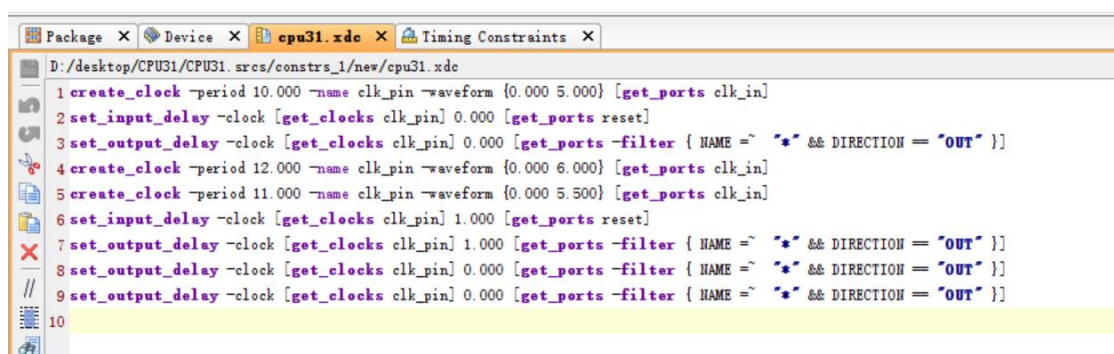
Command:

Reference Reset to Defaults OK Cancel

3.设置 Output Delay

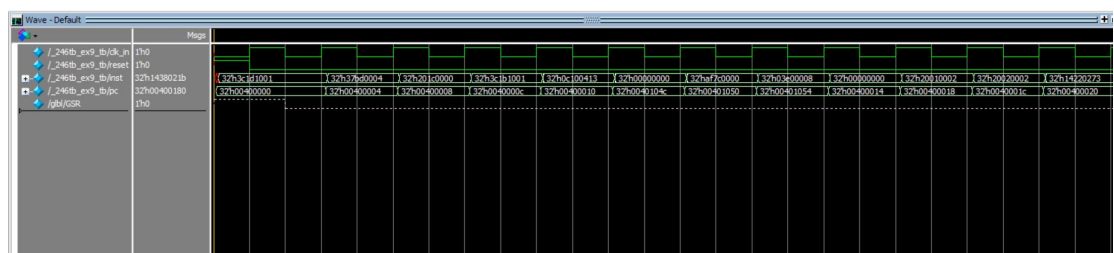


生成的 XDC 文件:



后仿真 modelsim 波形图:

可以看到相比于前仿真后仿真上面每一个指令都相较于 PC 后移了一些位置, 即 加入了门电路的延迟。



(2) 对时序分析的结果进行分析, 并截图。

设计时序摘要表(Design Timing Summary):

可以看到——"所有指定的时序约束均已满足", 这表明设计满足了所有时序要求, 没有违反时序约束的情况。

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (VNS):	0.110 ns	Worst Hold Slack (VHS):	0.147 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	3267	Total Number of Endpoints:	1089

All user specified timing constraints are met.

得到的时序报告（timing_report）：

```

timing_report.txt
文件  编辑  查看

Copyright 1986-2016 Xilinx, Inc. All Rights Reserved.

-----
Tool Version : Vivado v.2016.2 (win64) Build 1577090 Thu Jun  2 16:32:40 MDT 2016
Date        : Sun May 18 22:34:49 2025
Host        : LAPTOP-VOAHC9H running 64-bit major release (build 9200)
Command     : report_timing_summary -delay_type min_max -report_unconstrained -
check_timing_verbose -max_paths 10 -input_pins -name timing_1 -file D:/desktop/timing_report.txt
Design      : sccomp_dataflow
Device      : 7a100t-csg324
Speed File  : -1 PRODUCTION 1.14 2014-09-11
-----

Timing Summary Report

-----
Timer Settings
-----

Enable Multi Corner Analysis      : Yes
Enable Pessimism Removal          : Yes
Pessimism Removal Resolution      : Nearest Common Node
Enable Input Delay Default Clock  : No
Enable Preset / Clear Arcs        : No
Disable Flight Delays              : No
Ignore I/O Paths                  : No
Timing Early Launch at Borrowing Latches : false

Corner  Analyze  Analyze
Name    Max Paths Min Paths
-----
Slow    Yes     Yes
Fast    Yes     Yes

check_timing report

Table of Contents
-----
1. checking no_clock
2. checking constant_clock
3. checking pulse_width_clock

行 1, 列 1    357,375 个字符    100%    Windows (CRLF)    UTF-8

```

六、CPU 下板结果

(1) 对 CPU 下板的实验内容和结果进行描述，并给出下板结果的贴图。

下板时,需要将 PC 寄存器或通用寄存器结果输出到七段数码管上。为了实现这个流程，我新建了一个顶层模块 top，一个分频器模块 Divider，和一个七段数码管显示模块 seg7x16。

FPGA 系统时钟为 100MHz，人眼无法观察指令执行过程。所以我使用分频

器将时钟分频到 1Hz，方便人眼观察。

```
module Divider(  
    input clk,  
    input rst,  
    output reg clk_out  
);  
  
reg [31:0] counter = 0;  
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        counter <= 0;  
        clk_out <= 0;  
    end else begin  
        if (counter == 50000000) begin  
            clk_out <= ~clk_out;  
            counter <= 0;  
        end else begin  
            counter <= counter + 1;  
        end  
    end  
end  
endmodule
```

在顶层模块 top 中，我将 CPU 模块和分频器、七段数码管显示模块进行整合，100MHz 时钟驱动数码管的动态扫描，保证显示稳定不闪烁；1Hz 时钟控制指令执行的频率，也就是显示数据的更新频率，便于下板观察 CPU 指令执行的变化。

```
module top(  
    input  clk_in,          //时钟输入  
    input  reset,           //复位信号  
    output [7:0] o_seg,  
    output [7:0] o_sel  
);  
  
wire [31:0] inst;  
wire [31:0] pc;  
wire  clk_5hz;  
  
Divider divider_inst(  
    .clk(clk_in),  
    .rst(reset),  
    .clk_out(clk_1hz)  
);
```

```

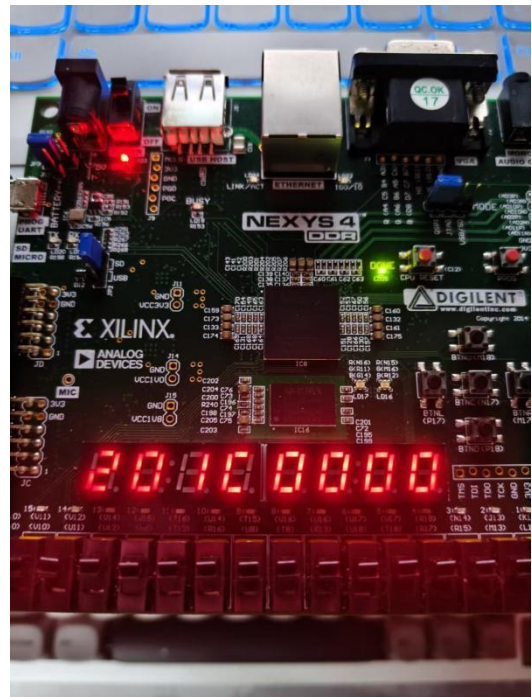
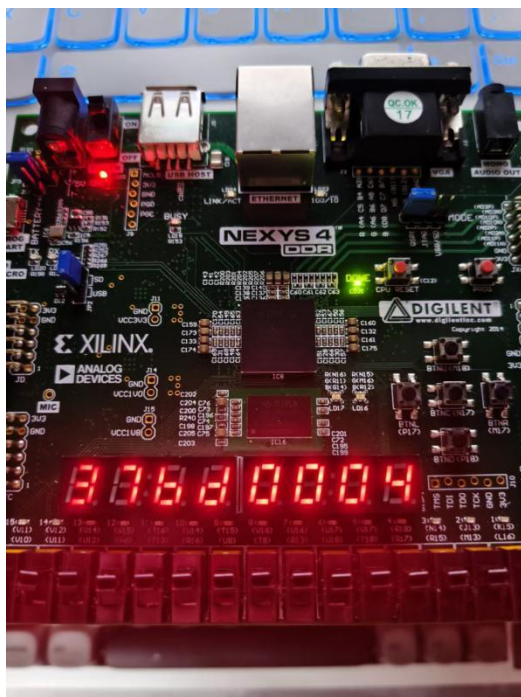
sccomp_dataflow sd(
    .clk_in(clk_1hz),
    .reset(reset),
    .inst(inst),
    .pc(pc)
);

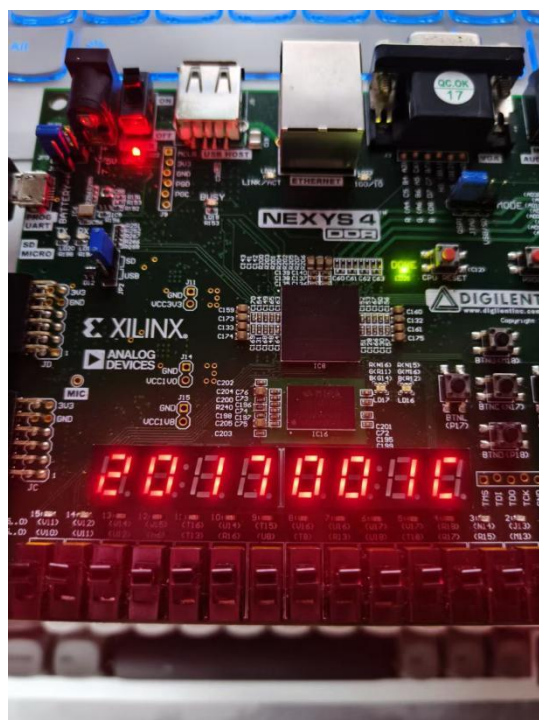
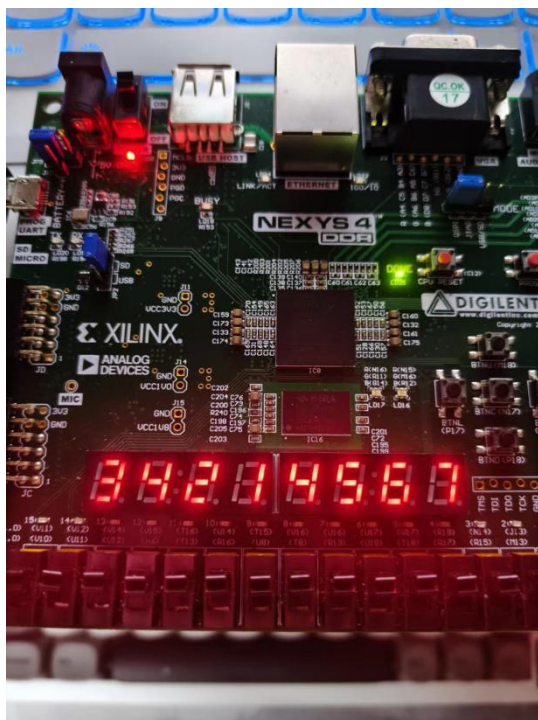
seg7x16 print(
    .clk(clk_in),
    .reset(reset),
    .cs(clk_1hz),
    .i_data(inst),
    .o_seg(o_seg),
    .o_sel(o_sel)
);

endmodule

```

下板结果：





(2) 实验的分析与结论。

在配置了 MIPS 网站提供的 board 下板 coe 文件后，下板后可以看到指令的执行过程，每秒钟执行一条指令。下板后一开始会循环执行 4 条指令，按下 reset 按键之后会开始执行内置指令，但最后还是会陷入 4 条指令的死循环。

七、心得体会及建议

本次设计 31 条指令 CPU 的实验让我大有收获。

首先就是我对 CPU 内部结构的理解更为深刻了，本次设计的 CPU 实际上是哈佛结构，而不是我们之前所学的冯诺依曼结构。冯诺依曼结构有一个重要的概念叫做存储程序，差不多就是“程序和数据同存在一个存储器中”。而哈佛结构不是这样，其将程序和数据分开存储，也就是使用两个存储器，一为程序存储器（IMEM），一为数据存储器（DMEM）。

数据通路设计是核心环节之一。首先，需要仔细分析每一条 MIPS 指令的功能，确定其在执行过程中需要用到的所有硬件部件，包括取指单元、寄存器堆、ALU 等。然后，将这些部件及其数据来源整理成表格，为后续画出每条指令的独立数据通路提供了清晰的依据。最关键的是将这 31 条指令的独立数据通路整合为一个统一的、能够支持所有指令的总数据通路。这个过程需要反复斟酌，确保路径的正确性和高效性，避免冲突。

紧接着是控制部件的设计。在已形成的数据通路基础上，我为每条指令绘制了从取指到执行的微指令流程图。依据这些流程图，我编排了指令各阶段的操作时序表，并由此推导出每个控制信号的逻辑表达式。最后，通过逻辑门电路来实现这些控制信号，完成了控制单元的设计。这一步让我深刻体会到，精确的控制是 CPU 正确执行指令的关键。

仿真测试是检验设计成果的重要手段。我先后进行了 CPU 的前仿真和后仿真测试。在前仿真阶段，我逐条测试了 31 条指令的基本功能，并进行了指令边界数据测试和随机指令序列测试。通过将 CPU 仿真结果与 MARS 模拟器的执行结果进行比对，有效地发现了设计中存在的 bug。后仿真阶段则更加接近实际硬件行为，特别需要注意的是，此时 CPU 代码中不能包含 `initial`、`$finish` 等不可综合语句。最后的下板测试环节，将设计真正部署到 FPGA 开发板上。

总的来说，本次实验不仅让我系统地掌握了单周期 CPU 的设计流程，从指令集分析、数据通路搭建、控制单元实现到仿真验证和板级测试，更锻炼了我分析问题、解决问题以及动手实践的能力。在调试过程中，我和室友们不断测试、对比，互相帮助找出每一条指令的错误。定位一个微小的逻辑错误往往需要花费大量的时间和精力，但成功排除故障后的成就感也是切实的。

通过这次实验，我对计算机组成的理论知识有了更直观和深入的理解，也为后续设计更复杂的 CPU 设计打下了坚实的基础和强有力的信心。